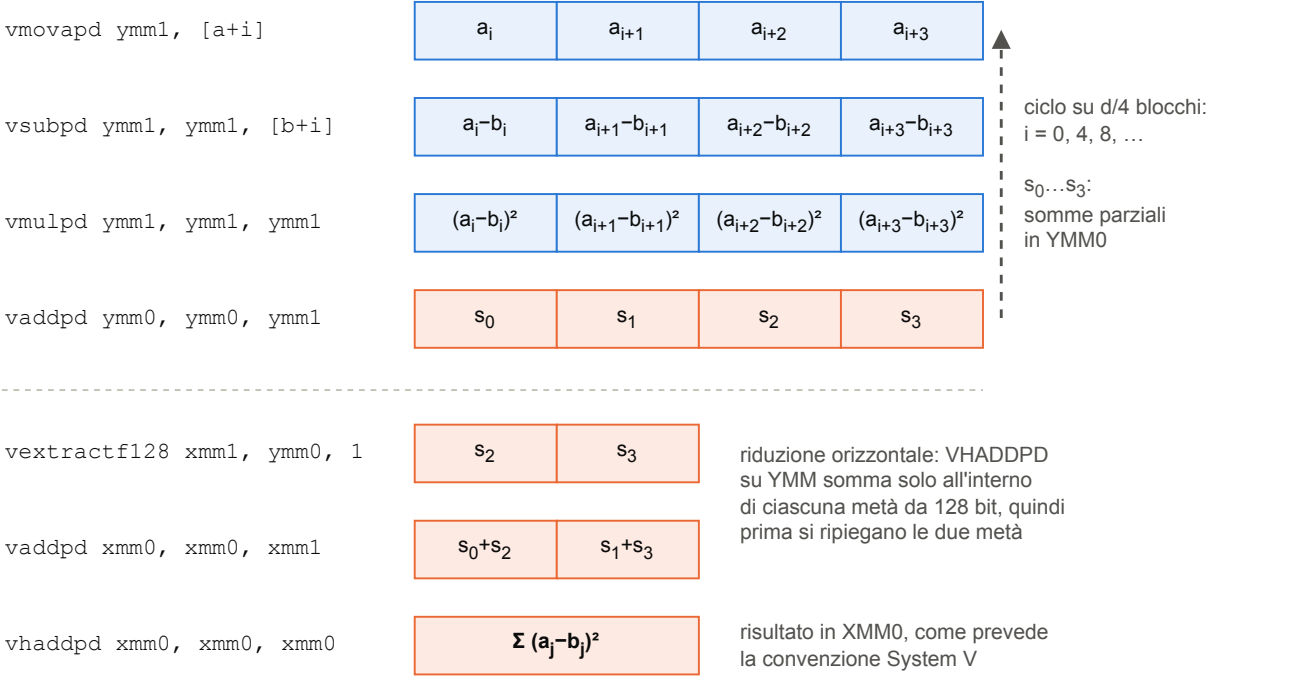

vmovapd ymm1, [a+i]
a_i a_{i+1} a_{i+2} a_{i+3}
vsubpd ymm1, ymm1, [b+i]
a_i-b_i $a_{i+1}-b_{i+1}$ $a_{i+2}-b_{i+2}$ $a_{i+3}-b_{i+3}$
vmulpd ymm1, ymm1, ymm1
$(a_i-b_i)^2$ $(a_{i+1}-b_{i+1})^2$ $(a_{i+2}-b_{i+2})^2$ $(a_{i+3}-b_{i+3})^2$
vaddpd ymm0, ymm0, ymm1
s_0 s_1 s_2 s_3
ciclo su d/4 blocchi:
i = 0, 4, 8, …
$s_0 \dots s_3$:
somme parziali
in YMM0
vextractf128 xmm1, ymm0, 1
s_2 s_3
vaddpd xmm0, xmm0, xmm1
s_0+s_2 s_1+s_3
riduzione orizzontale: VHADDPD
su YMM somma solo all'interno
di ciascuna metà da 128 bit, quindi
prima si ripiegano le due metà
vhaddpd xmm0, xmm0, xmm0
$\Sigma (a_j-b_j)^2$
risultato in XMM0, come prevede
la convenzione System V